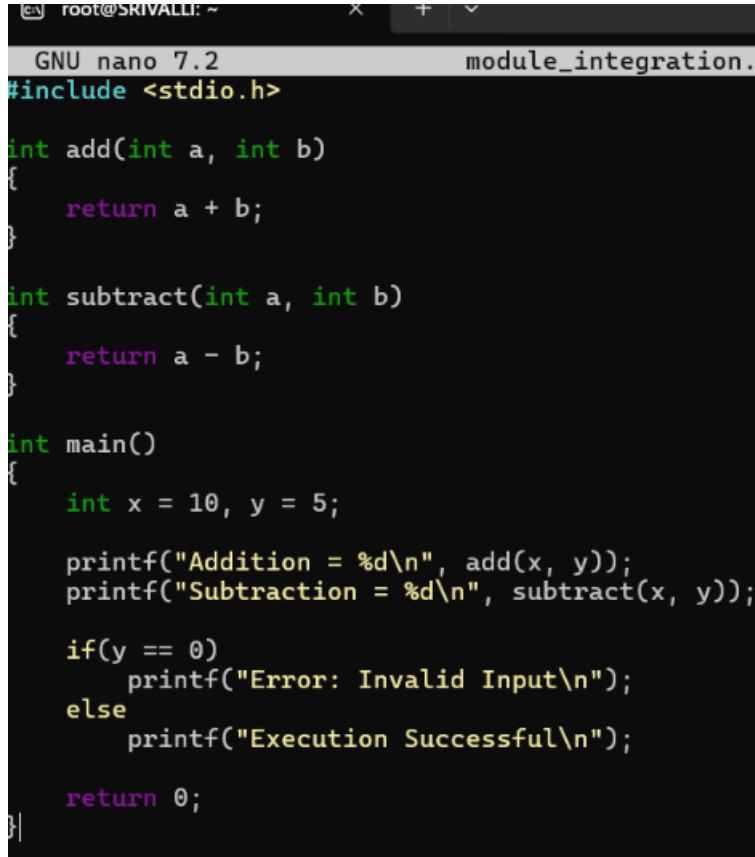


SKILL_11

1. To Connect Modules, Validate Interfaces, Test End-tc;-End Execution, Handle Errors, Improve Reliability, Refactor 4 Components

CODE:



```
GNU nano 7.2 module_integration.c
#include <stdio.h>

int add(int a, int b)
{
    return a + b;
}

int subtract(int a, int b)
{
    return a - b;
}

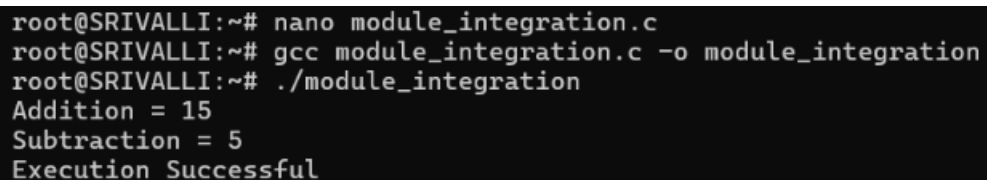
int main()
{
    int x = 10, y = 5;

    printf("Addition = %d\n", add(x, y));
    printf("Subtraction = %d\n", subtract(x, y));

    if(y == 0)
        printf("Error: Invalid Input\n");
    else
        printf("Execution Successful\n");

    return 0;
}
```

RESULT:



```
root@SRIVALLI:~# nano module_integration.c
root@SRIVALLI:~# gcc module_integration.c -o module_integration
root@SRIVALLI:~# ./module_integration
Addition = 15
Subtraction = 5
Execution Successful
```

REPORT:

In this experiment, different program modules were integrated and tested together. The interfaces between modules were validated, errors were handled, and the complete system execution was verified. The experiment helps in understanding module integration, reliability improvement, error handling, and software testing in Linux systems

2. To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios

CODE:

```
root@SRIVALLI: ~
GNU nano 7.2 error_handling.c *
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int a, b;

    printf("Enter two numbers: ");

    if(scanf("%d %d", &a, &b) != 2)
    {
        printf("Error: Invalid Input\n");
        return 1;
    }

    if(b == 0)
    {
        printf("Error: Division by Zero Not Allowed\n");
        return 1;
    }

    printf("Result = %d\n", a / b);

    return 0;
}
```

RESULT:

```
root@SRIVALLI:~# nano error_handling.c
root@SRIVALLI:~# gcc error_handling.c -o error_handling
root@SRIVALLI:~# ./error_handling
Enter two numbers: 66 4
Result = 16
```

REPORT:

In this experiment, a program was developed to detect runtime errors and handle invalid inputs. Error messages were generated for invalid syntax and division by zero conditions. The program recovered gracefully without crashing and displayed appropriate messages. This experiment helps in understanding error detection, error handling, and improving software reliability.

3. To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings

CODE:

```

GNU nano 7.2 memory_leak.c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *ptr;

    ptr = (int *)malloc(10 * sizeof(int));

    printf("Memory Allocated\n");

    /* Memory not freed intentionally */

    return 0;
}

```

RESULT:

```

root@SRIVALLI:~# nano memory_leak.c
root@SRIVALLI:~# gcc memory_leak.c -o memory_leak
root@SRIVALLI:~# sudo apt install valgrind
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
valgrind is already the newest version (1:3.22.0-0ubuntu3).
0 upgraded, 0 newly installed, 0 to remove and 27 not upgraded.
root@SRIVALLI:~# valgrind ./memory_leak
==1018== Memcheck, a memory error detector
==1018== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==1018== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright i
==1018== Command: ./memory_leak
==1018==
Memory Allocated
==1018==
==1018== HEAP SUMMARY:
==1018==    in use at exit: 40 bytes in 1 blocks
==1018==   total heap usage: 2 allocs, 1 frees, 1,064 bytes allocated
==1018==
==1018== LEAK SUMMARY:
==1018==    definitely lost: 40 bytes in 1 blocks
==1018==    indirectly lost: 0 bytes in 0 blocks
==1018==    possibly lost: 0 bytes in 0 blocks
==1018==    still reachable: 0 bytes in 0 blocks
==1018==    suppressed: 0 bytes in 0 blocks
==1018== Rerun with --leak-check=full to see details of leaked memory
==1018==
==1018== For lists of detected and suppressed errors, rerun with: -s
==1018== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
root@SRIVALLI:~# nano memory_leak.c
root@SRIVALLI:~# gcc memory_leak.c -o memory_leak

```

REPORT:

In this experiment, Valgrind was used to detect memory leaks in a C program. Memory allocated using malloc() was analyzed, and leaks were identified. After releasing the allocated memory using free(), Valgrind confirmed that no memory leaks remained. This experiment helps in understanding memory management, leak detection, and resource cleanup in Linux systems.

4. To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Result

CODE:

```
root@SRIVALLI: ~  
GNU nano 7.2 test  
#include <stdio.h>  
  
int main()  
{  
    int a, b;  
  
    printf("Enter two numbers: ");  
    scanf("%d %d", &a, &b);  
  
    printf("Sum = %d\n", a + b);  
  
    return 0;  
}  
  
root@SRIVALLI: ~  
GNU nano 7.2  
echo "10 20" | ./test_program  
echo "5 15" | ./test_program  
echo "100 200" | ./test_program
```

RESULT:

```
root@SRIVALLI:~# nano memory_leak.c  
root@SRIVALLI:~# gcc memory_leak.c -o memory_leak  
root@SRIVALLI:~# nano test_program.c  
root@SRIVALLI:~# gcc test_program.c -o test_program  
root@SRIVALLI:~# nano test.sh  
root@SRIVALLI:~# chmod +x test.sh  
root@SRIVALLI:~# ./test.sh  
Enter two numbers: Sum = 30  
Enter two numbers: Sum = 20  
Enter two numbers: Sum = 300  
root@SRIVALLI:~#
```

REPORT:

In this experiment, test cases were designed and executed automatically using a shell script. The outputs were verified and compared with expected results to ensure correct program behavior. This experiment helps in understanding test automation, output verification, and software testing in Linux systems.